

MASTER PROMPT — BUILD A COMPLETE LOCAL FOOD ORDERING WEBSITE

Act as a **senior full-stack software architect, UI/UX designer, database engineer, security engineer, QA engineer, and DevOps engineer**.

Your job is to create a complete, professional, production-ready **local food ordering e-commerce website** for a local food seller.

Do NOT create only a static UI mockup.

Build an actually functional application with:

- Customer registration and login
- Product/menu browsing
- Shopping cart
- Checkout
- UPI-only payment
- Order management
- Email confirmation
- Customer account area
- Complete admin dashboard
- Product CRUD
- Category management
- Inventory/availability management
- Customer management
- Order management
- Dynamic reports
- Predefined/canned reports
- Database persistence
- Authentication and authorization
- Security
- Responsive UI
- Dark neon visual theme
- Proper error handling
- Production-quality code structure

The application should be clean, maintainable, modular, scalable, and easy to deploy.

1. PROJECT OBJECTIVE

Create a website for a **local food seller** where customers can:

1. Visit the website.
2. Browse available food products.
3. Search and filter food items.
4. View product details.
5. Register/login.
6. Add items to a shopping cart.
7. Select quantities.
8. Enter delivery/contact information.
9. Place an order.
10. Pay using **UPI only**.
11. Receive an email when the booking/order is confirmed.
12. See their previous orders.
13. Track the status of current orders.

The seller/admin must have a separate secure administration interface where they can:

1. Add products.
2. Edit products.
3. Delete/archive products.
4. Add product images.
5. Change product prices.
6. Manage categories.
7. Mark products available/unavailable.
8. Manage inventory.
9. View orders.
10. Update order status.
11. View customers.
12. Search/filter orders.
13. View dashboard analytics.
14. Generate canned/predefined reports.
15. Build dynamic/custom reports.
16. Export reports.
17. Monitor payment status.
18. View business metrics.

2. RECOMMENDED TECHNOLOGY STACK

Use the following stack unless there is a strong technical reason not to.

Frontend

Use:

- Next.js latest stable version
- React
- TypeScript
- Tailwind CSS
- shadcn/ui or similarly professional accessible component library
- Lucide icons
- React Hook Form
- Zod for validation
- Recharts for dashboard charts

Use Next.js App Router.

Use server components where appropriate and client components only where interactivity requires them.

3. BACKEND

Use Next.js server-side functionality/API routes/server actions as appropriate.

Use:

- TypeScript
- Next.js backend APIs
- Prisma ORM
- PostgreSQL-compatible CockroachDB
- Zod validation
- Secure authentication
- Transaction-safe order creation

Do not trust values sent by the browser.

Prices, totals, product availability, discounts and payment amounts must always be recalculated/validated on the server.

4. DATABASE

The database is CockroachDB/PostgreSQL compatible.

Use Prisma ORM with PostgreSQL provider.

The database URL structure is:

```
postgresql://srikanta:<ENTER-SQL-USER-PASSWORD>@dogged-ant-20102.jxf.gcp-asia-south1.cockroachlabs.cloud:26257/defaultdb?sslmode=verify-full
```

IMPORTANT SECURITY REQUIREMENT:

NEVER hard-code this connection string or database password inside:

- JavaScript files
- TypeScript files
- React components
- API routes
- Git repository
- public frontend configuration

Instead create:

`.env`

and use:

```
DATABASE_URL="postgresql://srikanta:<ENTER-SQL-USER-PASSWORD>@dogged-ant-20102.jxf.gcp-asia-south1.cockroachlabs.cloud:26257/defaultdb?sslmode=verify-full"
```

The actual SQL password will be manually entered into the environment variable.

Add `.env` and `.env.local` to `.gitignore`.

Create `.env.example` containing placeholders only.

Example:

```
DATABASE_URL= AUTH_SECRET= NEXTAUTH_URL= EMAIL_HOST= EMAIL_PORT= EMAIL_USER=
EMAIL_PASSWORD= EMAIL_FROM= PAYMENT_PROVIDER_KEY_ID= PAYMENT_PROVIDER_KEY_SECRET=
PAYMENT_WEBHOOK_SECRET= NEXT_PUBLIC_APP_URL=
```

Never expose private environment variables using a `NEXT_PUBLIC_` prefix.

5. APPLICATION ROLES

Implement role-based access control.

Minimum roles:

CUSTOMER

Normal registered website user.

Can:

- Browse products
- Manage profile
- Manage addresses
- Add items to cart
- Place orders
- Pay
- View only their own orders

ADMIN

Administrator.

Can:

- Access `/admin`
- Manage products
- Manage categories
- Manage orders
- Manage customers
- View reports
- View analytics
- Manage site settings

Architect the authorization system so additional roles can later be added, such as:

- MANAGER
- DELIVERY_STAFF
- KITCHEN_STAFF

Never depend only on hiding frontend buttons.

Every admin API endpoint/server action must verify authorization on the server.

6. AUTHENTICATION

Implement secure authentication.

Support:

- Customer registration
- Login
- Logout
- Password hashing
- Secure session handling
- Forgot password
- Password reset
- Email uniqueness validation
- Protected customer routes
- Protected admin routes

Recommended:

- Auth.js / NextAuth or a secure equivalent.

Passwords must be hashed using:

- Argon2 or
- bcrypt

Never store plain-text passwords.

Registration fields:

- Full name
- Email
- Mobile number
- Password
- Confirm password

Optionally allow:

- WhatsApp number

Validation:

- Valid email
 - Valid Indian mobile number
 - Strong password
 - Password confirmation match
-

7. DATABASE DESIGN

Create a normalized database schema.

At minimum include the following entities.

User

Fields:

- id
- name
- email
- phone
- passwordHash
- role
- emailVerified
- isActive
- createdAt
- updatedAt
- lastLoginAt

Use an enum for role.

Address

Fields:

- id
- userId
- label
- recipientName
- phone
- addressLine1
- addressLine2
- landmark
- city
- state
- postalCode
- latitude optional
- longitude optional
- isDefault
- createdAt
- updatedAt

A customer can maintain multiple addresses.

Examples:

- Home
- Office
- Other

Category

Fields:

- id
- name
- slug
- description
- imageUrl
- displayOrder
- isActive
- createdAt
- updatedAt

Product

Fields:

- id
- categoryId
- name
- slug
- shortDescription
- description
- sku
- price
- compareAtPrice optional
- imageUrl
- thumbnailUrl optional
- isVegetarian
- isVegan optional
- isSpicy optional
- spiceLevel optional
- preparationTimeMinutes
- stockQuantity optional
- trackInventory

- minimumOrderQuantity
- maximumOrderQuantity optional
- isAvailable
- isFeatured
- displayOrder
- createdAt
- updatedAt
- deletedAt optional

Use decimal/numeric database type for currency.

Never store money as floating-point numbers.

ProductImage

Allow multiple product images.

Fields:

- id
 - productId
 - imageUrl
 - altText
 - displayOrder
 - createdAt
-

Cart

Fields:

- id
 - userId
 - createdAt
 - updatedAt
-

CartItem

Fields:

- id
- cartId
- productId

- quantity
 - createdAt
 - updatedAt
-

Order

Fields:

- id
- orderNumber
- userId
- addressId optional
- customerName
- customerEmail
- customerPhone
- deliveryAddressSnapshot
- subtotal
- deliveryCharge
- discountAmount
- taxAmount
- grandTotal
- paymentStatus
- orderStatus
- customerNotes
- adminNotes
- createdAt
- confirmedAt
- preparingAt
- readyAt
- dispatchedAt
- deliveredAt
- cancelledAt
- updatedAt

The delivery address must be stored as a snapshot on the order so future customer address modifications do not modify historical orders.

Generate easy-to-read order numbers such as:

FOOD-20260830-000123

Do not expose sequential database IDs as the primary public order reference.

OrderItem

Fields:

- id
- orderId
- productId optional
- productNameSnapshot
- productSkuSnapshot
- unitPrice
- quantity
- lineTotal
- createdAt

Store product name and price snapshots.

Historical order information must remain correct even if a product is later edited.

Payment

Fields:

- id
- orderId
- provider
- providerOrderId
- providerPaymentId
- providerSignature
- upiTransactionId optional
- amount
- currency
- status
- paymentMethod
- failureReason
- rawProviderResponse optional JSON
- createdAt
- updatedAt
- paidAt

Currency:

INR

Payment method:

UPI

OrderStatusHistory

Fields:

- id
- orderId
- oldStatus
- newStatus
- changedByUserId
- notes
- createdAt

Keep a complete status audit trail.

AdminAuditLog

Fields:

- id
- adminUserId
- action
- entityType
- entityId
- beforeData optional JSON
- afterData optional JSON
- ipAddress
- userAgent
- createdAt

Record important admin actions such as:

- product creation
 - product price changes
 - deletion
 - order cancellation
 - status modification
 - customer activation/deactivation
-

8. ORDER STATUS ENUM

Implement statuses similar to:

PENDING_PAYMENT

PAYMENT_RECEIVED

CONFIRMED

PREPARING

READY

OUT_FOR_DELIVERY

DELIVERED

CANCELLED

REFUND_PENDING

REFUNDED

The exact statuses may be adjusted if required, but keep them clearly defined.

Show user-friendly names on the frontend.

9. PAYMENT STATUS ENUM

Use:

PENDING

PROCESSING

PAID

FAILED

CANCELLED

REFUNDED

PARTIALLY_REFUNDED

10. UPI PAYMENT

The website must support **UPI payment only**.

Do not show:

- Credit card
- Debit card
- Net banking
- Cash on delivery

For production architecture, use a reputable Indian payment gateway that supports UPI, such as:

- Razorpay
- Cashfree
- PhonePe Payment Gateway

Design the payment system behind a provider/service abstraction so the gateway can be replaced later.

Example architecture:

PaymentService

Functions:

createPaymentOrder()

verifyPayment()

processWebhook()

getPaymentStatus()

refundPayment()

Prefer Razorpay UPI initially if implementation examples are required.

IMPORTANT:

Do not consider the payment successful merely because the browser redirects to a success page.

Payment must be verified server-side using:

- payment provider signature verification and/or

- secure payment webhook

The order should only move to payment-confirmed status after verified payment.

Implement webhook endpoint.

Example:

POST /api/webhooks/payment

Verify webhook signature before processing.

Prevent duplicate webhook processing.

Implement idempotency.

If payment fails:

- keep order identifiable
- show payment failure
- allow retry if appropriate

Do not create duplicate confirmed orders when the user retries payment.

11. OPTIONAL DEVELOPMENT PAYMENT MODE

Because actual gateway credentials may not initially be available, create a development/mock payment provider.

Environment variable example:

PAYMENT_MODE=mock

or

PAYMENT_MODE=razorpay

The mock provider must only be available in development.

Never enable mock-payment success in production.

12. EMAIL CONFIRMATION

Send an email when an order is successfully confirmed.

Email must be sent only after verified payment/confirmed order.

Support an email provider abstraction.

Allow SMTP configuration or providers such as:

- Resend
- SendGrid
- Amazon SES

Configuration through environment variables only.

Confirmation email should contain:

- Customer name
- Order number
- Order date/time
- Ordered products
- Quantities
- Item prices
- Subtotal
- Delivery charge
- Discount
- Total amount paid
- Payment status
- Delivery address
- Customer phone
- Expected preparation/delivery information
- Order tracking link
- Seller contact information

Use a professional HTML email template.

Example subject:

Order Confirmed — #FOOD-20260830-000123

Email failures must NOT incorrectly mark the payment/order as failed.

Record email delivery attempts/logs.

13. OPTIONAL EMAIL LOG TABLE

Create:

EmailLog

Fields:

- id
 - userId optional
 - orderId optional
 - recipient
 - subject
 - template
 - status
 - providerMessageId
 - errorMessage optional
 - sentAt optional
 - createdAt
-

14. HOME PAGE

Create a visually impressive homepage.

Sections:

Header

Include:

- Logo
- Home
- Menu
- Categories
- About
- Contact
- Cart
- Account/Login

Desktop and mobile navigation.

Sticky navigation is preferred.

Hero Section

Large dark neon hero area.

Include:

- Strong headline
- Supporting text
- CTA button such as "Order Now"
- Secondary CTA "View Menu"
- High-quality food presentation area

Example headline:

"Fresh Local Food. Bold Flavours. Delivered Near You."

Do not rely on this exact wording if you can produce something better.

Featured Categories

Cards for categories such as:

- Breakfast
- Snacks
- Meals
- Sweets
- Beverages

Categories must come dynamically from the database.

Featured Products

Display products marked `isFeatured = true`.

Product cards should show:

- Product image
 - Product name
 - Short description
 - Vegetarian indicator if applicable
 - Price
 - Availability
 - Add to Cart
 - Quantity controls
-

Why Choose Us

Possible attributes:

- Freshly prepared
- Local ingredients
- Secure UPI payments
- Quick local delivery
- Hygienic packaging

Make the content editable later.

Footer

Include:

- Brand/logo
 - Contact details
 - Address
 - Phone
 - Email
 - Social media links
 - Privacy Policy
 - Terms & Conditions
 - Refund/Cancellation Policy
-

15. MENU / PRODUCTS PAGE

Route:

/menu

Features:

- Product grid
- Search
- Category filter
- Vegetarian filter
- Availability filter
- Sort options
- Price sorting
- Featured products
- Pagination or infinite loading if required

Search should support product:

- name
- description
- category

Keep query filters in URL parameters where practical.

Example:

/menu?category=snacks&sort=price-asc

16. PRODUCT DETAILS PAGE

Route:

/product/[slug]

Display:

- Large image
- Image gallery
- Product name
- Price
- Description
- Category
- Vegetarian indicator
- Spice information if relevant
- Availability
- Preparation time
- Quantity selector
- Add to Cart
- Related products

Disable Add to Cart when the product is unavailable.

17. SHOPPING CART

Route:

/cart

Features:

- List cart items
- Product thumbnail
- Name
- Price
- Quantity selector
- Increase/decrease
- Remove
- Per-item total
- Cart subtotal
- Delivery charge estimate
- Grand total
- Checkout button

Always recalculate price on the server before order creation.

Never trust totals stored only in browser state.

Persist carts for logged-in customers.

18. CHECKOUT

Route:

/checkout

Require login.

Checkout should contain:

Customer information

- Name
- Email
- Phone

Delivery address

Allow:

- Select existing saved address
- Add new address

Order summary

Show:

- Products
- quantities
- prices
- subtotal
- delivery charge
- discount
- final amount

Customer notes

Example:

"Less spicy" "Call before delivery"

Do not automatically promise that all notes can be fulfilled.

Payment

Only show:

UPI Payment

Display secure payment wording.

Button:

"Pay ₹XXX with UPI"

19. ORDER SUCCESS PAGE

Route:

/order/success/[orderNumber]

Display:

- Success/confirmation icon
- Order number
- Amount
- Payment status

- Order status
- Ordered products
- Delivery address
- Estimated time if configured
- "View My Orders"
- "Continue Shopping"

Do not display success unless server confirms that the order exists and belongs to the authenticated customer.

20. CUSTOMER DASHBOARD

Routes:

/account

/account/profile

/account/addresses

/account/orders

/account/orders/[orderNumber]

Features:

Account overview

- Name
- Email
- Phone
- Recent orders

Profile

Allow customer to edit:

- Name
- Phone

Handle email changes securely.

Addresses

- Add

- Edit
- Delete
- Make default

My Orders

Display:

- Order number
- Date
- Total
- Payment status
- Order status
- View details

Filters:

- All
 - Active
 - Delivered
 - Cancelled
-

21. ORDER TRACKING

Customer order details should show visual status progress.

Example timeline:

Order Placed ↓ Payment Received ↓ Confirmed ↓ Preparing ↓ Ready ↓ Out for Delivery ↓ Delivered

Use icons and timestamps.

22. ADMIN LOGIN AND SECURITY

Admin dashboard:

/admin

Admin routes must require authenticated ADMIN role.

If a normal customer manually enters `/admin`, deny access.

Do not rely on frontend checks alone.

Use server-side role authorization.

23. ADMIN DASHBOARD

Route:

/admin

Create a professional dashboard.

Top KPI cards:

- Today's Revenue
- Today's Orders
- Pending Orders
- Preparing Orders
- Delivered Orders
- Total Customers
- Average Order Value
- Total Revenue

Add comparisons where practical:

- Today vs yesterday
- This week vs previous week
- This month vs previous month

Charts:

Sales Revenue

Selectable timeframe:

- Today
- 7 days
- 30 days
- 3 months
- 6 months
- 1 year
- Custom range

Order Status Distribution

Pie/donut chart.

Orders Over Time

Line or bar chart.

Top Selling Products

Bar chart/table.

Recent Orders

Display latest orders.

24. ADMIN SIDEBAR

Admin navigation:

Dashboard

Orders

Products

Categories

Customers

Reports

Payments

Settings

Audit Logs

Logout

Use collapsible mobile-friendly sidebar.

25. PRODUCT MANAGEMENT

Route:

/admin/products

Display data table.

Columns:

- Image
- Product
- SKU
- Category
- Price
- Availability
- Inventory
- Featured
- Updated
- Actions

Features:

- Search
- Category filter
- Status filter
- Sort
- Pagination
- Add Product
- Edit Product
- Archive/delete product
- Toggle availability
- Toggle featured

Product deletion needs confirmation.

Prefer soft deletion/archiving when the product exists in historical orders.

Never destroy historical order details.

26. ADD / EDIT PRODUCT

Create professional forms.

Fields:

- Product name
- Slug
- SKU

- Category
- Short description
- Full description
- Price
- Compare-at price
- Product images
- Vegetarian
- Vegan
- Spicy
- Spice level
- Preparation time
- Track inventory
- Stock quantity
- Min quantity
- Max quantity
- Available
- Featured
- Display order

Validate all fields server-side.

27. IMAGE UPLOAD

Implement image upload architecture.

Do not store massive image binary data directly in PostgreSQL.

Use object storage such as:

- Cloudinary
- AWS S3
- Supabase Storage

or an interchangeable storage abstraction.

During development, provide a simple local placeholder mode if cloud credentials are unavailable.

Validate:

- Image MIME types
- File extension
- File size

Recommended max image size:

5 MB

Generate/use optimized images.

Use Next.js Image component.

28. CATEGORY MANAGEMENT

Route:

/admin/categories

Features:

- Add category
- Edit category
- Delete/archive
- Change display order
- Activate/deactivate
- Upload category image

Prevent destructive deletion when it would break product relationships.

29. ORDER MANAGEMENT

Route:

/admin/orders

Use a table.

Columns:

- Order number
- Customer
- Phone
- Date
- Items
- Amount
- Payment
- Status
- Actions

Filters:

- Order status
- Payment status
- Date range
- Customer
- Order number

Search by:

- Order number
 - Customer name
 - Email
 - Phone
-

30. ADMIN ORDER DETAILS

Route:

/admin/orders/[orderNumber]

Display:

- Order information
- Customer
- Phone
- Email
- Delivery address
- Products
- Quantity
- Item prices
- Subtotal
- Delivery charge
- Tax
- Discount
- Grand total
- Payment details
- Order notes
- Admin notes
- Status history

Allow authorized admin to change order status.

Example flow:

CONFIRMED → PREPARING → READY → OUT_FOR_DELIVERY → DELIVERED

Require confirmation for cancellation/refund-related actions.

Store every status change in OrderStatusHistory.

31. CUSTOMER MANAGEMENT

Route:

/admin/customers

Display:

- Name
- Email
- Phone
- Registration date
- Number of orders
- Lifetime spend
- Last order
- Status

Admin can:

- Search
- View profile
- View customer's order history
- Activate/deactivate customer

Do not expose password hashes.

32. PAYMENT MANAGEMENT

Route:

/admin/payments

Show:

- Payment ID
- Order number
- Customer

- Provider
- Provider transaction ID
- Amount
- Method
- Status
- Timestamp

Filters:

- Date
- Payment status
- Order
- Customer

Admin should be able to inspect payment status but never expose gateway secret keys.

33. REPORTING SYSTEM

Reporting is an important requirement.

Build both:

1. Canned/predefined reports
 2. Dynamic/custom reports
-

34. CANNED REPORTS

Create reports including:

Daily Sales Report

Fields:

- Date
- Order count
- Gross sales
- Discounts
- Delivery charges
- Net sales
- Average order value

Weekly Sales Report

Aggregate per week.

Monthly Sales Report

Aggregate per month.

Product Sales Report

Fields:

- Product
- Quantity sold
- Sales amount
- Number of orders

Top Selling Products

Rank by:

- Quantity or
- Revenue

Category Sales Report

Fields:

- Category
- Quantity
- Revenue

Order Status Report

Number and value of orders by status.

Payment Report

Payment totals grouped by:

- Date
- Status

Customer Report

Include:

- Customer name
- registration date
- order count
- lifetime order value

- last purchase

Top Customers

Rank by:

- Total amount spent
- Number of orders

Cancelled Order Report

Fields:

- Order
- Customer
- Amount
- Cancellation date
- cancellation reason if present

Tax Report

If taxes are enabled.

Delivery Charge Report

Show delivery charge totals.

35. DYNAMIC REPORT BUILDER

Create:

/admin/reports/custom

The administrator must be able to create reports by selecting:

Date range

- Today
- Yesterday
- Last 7 days
- Last 30 days
- This month
- Previous month
- This year
- Custom start/end dates

Metrics

Examples:

- Total revenue
- Net revenue
- Order count
- Average order value
- Quantity sold
- Discount
- Delivery charge

Dimensions/grouping

Examples:

- Date
- Day
- Week
- Month
- Product
- Category
- Customer
- Order status
- Payment status

Filters

- Product
- Category
- Customer
- Order status
- Payment status
- Minimum order value
- Maximum order value

The reporting queries must be generated safely.

Never concatenate arbitrary SQL directly from frontend input.

Use an allow-list of supported dimensions, metrics, filters and server-controlled query generation.

36. REPORT VISUALIZATION

Reports should support:

- Table view
- Line chart
- Bar chart
- Pie chart where applicable

Charts should be responsive and accessible.

Use Recharts.

Allow table sorting.

37. REPORT EXPORTS

Allow export to:

- CSV

Preferably also:

- Excel/XLSX
- PDF

Exports must contain the report filters/date range.

Generate exports server-side.

For very large reports, use pagination/streaming or appropriate server-side processing.

38. DELIVERY CONFIGURATION

Create application settings supporting:

- Store name
- Store address
- Contact phone
- Contact email
- UPI merchant/payment provider configuration
- Default preparation time

- Delivery enabled/disabled
- Pickup enabled/disabled
- Minimum order amount
- Delivery fee
- Free delivery threshold
- Serviceable postal codes

Do not store payment secrets in a normal database settings table unless encrypted.

Secrets should remain environment variables/secret manager entries.

39. LOCAL DELIVERY RULES

Since this is a local food seller, include ability to define serviceable areas.

Simplest initial implementation:

Allowed postal codes / PIN codes.

On checkout:

- Validate customer postal code.
- Reject unsupported areas politely.

Example:

"Sorry, delivery is currently unavailable for PIN code 560001."

Make delivery configuration adjustable by admin later.

40. BUSINESS HOURS

Create optional business hours configuration.

Fields per day:

- Open/closed
- Opening time
- Closing time

If ordering is unavailable outside business hours, show:

"Currently closed. Ordering opens at 8:00 AM."

Allow admin to configure whether future/pre-orders are accepted.

41. INVENTORY

Products can be:

Unlimited availability

For freshly prepared foods.

or

Inventory tracked

Example:

50 sweet boxes available.

When stock tracking is enabled:

- Validate stock before payment/order confirmation.
- Prevent negative inventory.
- Handle concurrent orders safely.
- Update inventory within database transactions.

If stock changes during checkout, explain clearly to customer.

42. DISCOUNTS — FUTURE-READY

Even if discounts are not initially enabled, design schema/service architecture so these can later be added:

- Coupon codes
- Fixed discount
- Percentage discount
- Minimum cart value
- Start/end dates
- Usage limits

Do not overcomplicate the initial implementation, but avoid architectural blockers.

43. DARK + NEON UI DESIGN

The website must have a premium **dark neon food-tech aesthetic**.

Avoid making it look like a generic Bootstrap/admin template.

Use a coherent design system.

Background

Use shades such as:

• **050505**

• **080A0C**

• **0B0E11**

• **111318**

Primary Neon Accent

Use a neon lime/green similar to:

• **A3FF12**

or

• **B6FF2E**

Secondary Neon Accent

Use sparingly:

- Cyan
- Electric turquoise

Example:

00F5D4

Error

Use a clear warm neon red/coral.

Warning

Use amber.

Text:

Primary text should be near-white.

Secondary text should have sufficient contrast.

44. DESIGN STYLE

Use:

- Large bold typography
- Rounded cards
- Subtle glassmorphism where appropriate
- Thin borders
- Neon glow accents
- Gradient glow backgrounds
- Sophisticated shadows
- Modern food photography
- Smooth hover transitions
- Subtle animations

Do NOT:

- Apply glow to every element
- Make text difficult to read
- use excessive animation
- use extremely saturated colors everywhere

The design should feel premium rather than gimmicky.

45. PRODUCT CARDS

Product cards should look polished.

Include:

- Image
- Category
- Product title
- Small description
- Veg/non-veg indicator
- Price
- Availability
- Add-to-cart CTA

Use subtle neon border/glow on hover.

Use micro-interaction when item is successfully added to cart.

46. ADMIN DESIGN

Admin interface should use the same brand language but be more functional.

Use:

- Dark dashboard background
- Neon highlights
- Clear data tables
- KPI cards
- Search/filter toolbars
- Charts
- Modal confirmation dialogs
- Toast notifications
- Skeleton loading

Admin UI should prioritize clarity over decorative effects.

47. MOBILE RESPONSIVENESS

Design mobile-first.

Test conceptual layouts at:

- 320px
- 375px
- 390px
- 430px
- 768px
- 1024px
- 1280px
- 1440px+

On mobile:

- Navigation becomes drawer
- Admin sidebar becomes drawer
- Tables become horizontally scrollable or responsive cards
- Checkout remains easy to use
- Payment CTA remains prominent

Do not create desktop-only pages.

48. ACCESSIBILITY

Follow WCAG principles.

Include:

- Semantic HTML
- Keyboard navigation
- Visible focus states
- Labels for form inputs
- ARIA only where needed
- Proper button elements
- Image alt text
- Sufficient color contrast

Do not rely on neon color alone to communicate state.

Use icons/text as well.

49. FORM VALIDATION

Every form requires:

Client validation

For fast feedback.

AND

Server validation

For security.

Use Zod schemas shared where appropriate.

Display errors next to fields.

Examples:

"Please enter a valid mobile number."

"Price must be greater than zero."

Avoid generic "Something went wrong" when a more useful safe message can be displayed.

50. SECURITY REQUIREMENTS

Implement security from the beginning.

Required:

- Password hashing
- Secure session cookies
- Authorization checks
- CSRF protection where relevant
- Input validation
- Output escaping
- SQL injection protection
- Rate limiting
- Secure HTTP headers
- Content Security Policy where practical
- Safe file upload validation
- Payment signature verification
- Webhook authentication
- Environment secret management

Never use SQL made from untrusted string concatenation.

Use Prisma queries or parameterized SQL.

51. RATE LIMITING

Apply rate limits especially to:

- Login
- Register
- Forgot password
- Password reset
- Checkout
- Payment initiation
- Webhooks where appropriate

Implement sensible IP/user-based controls.

52. DATABASE TRANSACTIONS

Use transactions for important workflows.

Particularly:

Order creation

Payment confirmation

Inventory reduction

Order status updates where multiple tables are affected

Avoid partially created orders.

53. IDEMPOTENCY

Critical operations must be idempotent.

Especially:

- Payment webhooks
- Order confirmation

- Email-triggering events where possible
- Payment retry logic

Receiving the same payment webhook twice must NOT:

- create two payments
 - create two orders
 - reduce stock twice
 - send excessive duplicate confirmations
-

54. ERROR LOGGING

Implement structured logging.

Log:

- Server errors
- Payment errors
- Webhook errors
- Email failures
- Important admin actions

Never log:

- Passwords
 - Authentication secrets
 - Database password
 - Payment gateway secrets
 - Full sensitive payment credentials
-

55. LOADING STATES

Use:

- Skeleton loaders
- Loading buttons
- Progress indicators

Prevent checkout button double-clicks.

When placing an order/payment:

Disable duplicate submissions while request is processing.

56. EMPTY STATES

Design attractive empty states.

Examples:

Cart empty:

"Your cart is hungry."

No orders:

"You haven't placed an order yet."

No search results:

"No dishes match your search."

Include relevant CTA.

57. ERROR PAGES

Create:

404 page

500/error UI

Unauthorized page

Payment failed page

Order not found state

Use consistent dark/neon styling.

58. SEO

Implement:

- Page titles
- Meta descriptions
- OpenGraph metadata
- Product metadata
- Sitemap
- robots.txt
- Semantic markup

Use structured data where appropriate for local business/products.

Do not expose admin routes in sitemap.

59. PERFORMANCE

Optimize:

- Images
- Database queries
- JavaScript bundles
- Fonts
- server rendering
- caching

Use indexes on commonly queried columns.

Suggested indexes:

User.email

User.phone

Product.slug

Product.categoryId

Product.isAvailable

Product.isFeatured

Order.orderNumber

Order.userId

Order.createdAt

Order.orderStatus

Order.paymentStatus

Payment.orderId

Payment.providerPaymentId

Category.slug

60. DATABASE MIGRATIONS

Create proper Prisma schema and migrations.

Include:

- schema.prisma
- migration instructions
- seed script

Do NOT advise manually editing production database tables.

61. DATABASE SEED

Create development seed data.

Categories example:

- Breakfast
- Snacks
- Main Course
- Sweets
- Beverages

Add 10–15 sample products.

Create an admin user using environment-controlled seed credentials.

Example environment variables:

SEED_ADMIN_EMAIL= SEED_ADMIN_PASSWORD=

Hash the password before storing it.

Never place a real admin password directly in Git.

62. API / SERVER ARCHITECTURE

Structure APIs cleanly.

Possible endpoints:

/api/auth/*

/api/products

/api/products/[id]

/api/categories

/api/cart

/api/cart/items

/api/checkout

/api/orders

/api/orders/[orderNumber]

/api/payments/create

/api/payments/verify

/api/webhooks/payment

/api/admin/products

/api/admin/orders

/api/admin/customers

/api/admin/reports

Use REST or server actions consistently.

Do not unnecessarily create APIs for operations that are better implemented as secure server actions.

63. SERVICE LAYER

Keep business logic separate from UI.

Recommended service structure:

services/ auth.service.ts product.service.ts category.service.ts cart.service.ts order.service.ts
payment.service.ts email.service.ts report.service.ts inventory.service.ts

Repositories/database abstractions can be introduced where they improve maintainability.

64. PROJECT STRUCTURE

Use a clean folder architecture similar to:

app/ (store)/ page.tsx menu/ product/ cart/ checkout/ account/ admin/ page.tsx products/ categories/
orders/ customers/ payments/ reports/ settings/ api/

components/ ui/ layout/ products/ cart/ checkout/ admin/ charts/

lib/ auth/ db/ validations/ utilities/

services/

prisma/ schema.prisma seed.ts

emails/

types/

public/

middleware.ts if needed

Use a structure appropriate to current Next.js best practices.

65. CODING RULES

Use strict TypeScript.

Avoid `any`.

Create reusable types.

Create reusable UI components.

Do not put the entire application in a single file.

Do not place database calls directly throughout random React components.

Avoid duplicated business logic.

Use readable naming.

Keep functions reasonably small.

Add explanatory comments only where necessary.

Do not clutter code with obvious comments.

66. DATE AND TIME

Store timestamps in UTC.

Display to customers/admin in Indian timezone:

Asia/Kolkata

Use appropriate date formatting.

Examples:

30 Aug 2026, 9:45 AM

Handle timezone conversions safely.

67. CURRENCY

Currency:

INR

Display prices using Indian rupee formatting.

Example:

₹299

₹1,249.00 if decimals are required.

Perform monetary calculations with Decimal types, not JavaScript floating-point calculations where accuracy could be affected.

68. NOTIFICATIONS

Use toast notifications for interactions.

Examples:

"Product added to cart."

"Product updated successfully."

"Order status changed to Preparing."

"Payment could not be verified."

Do not expose internal stack traces in user notifications.

69. CONFIRMATION DIALOGS

Require confirmation for destructive operations such as:

- Delete/archive product
- Delete category
- Cancel order
- Deactivate customer

Example:

"Archive Paneer Meal?"

"This item will no longer appear to customers. Existing order history will not be affected."

70. ADMIN REPORT DASHBOARD DETAILS

Report page should have:

Top filter bar

- Start date
- End date
- Status
- Category
- Product
- Customer

KPI row

- Revenue
- Orders
- Units sold
- Average order value

Charts

- Revenue trend
- Order trend
- Product performance

Table

- Detailed report rows

Buttons

- Export CSV
- Export XLSX
- Export PDF if implemented

Use database-side aggregation instead of downloading all records to browser and aggregating there.

71. CANNED REPORT DATE FILTERS

Every applicable report should provide:

Today

Yesterday

This Week

Last Week

This Month

Last Month

Last 30 Days

This Year

Custom Range

72. ADMIN DASHBOARD REAL-TIME EXPERIENCE

Orders don't have to use WebSockets initially.

Implement periodic refresh or server revalidation.

Architect it so real-time updates using WebSockets/SSE can later be introduced.

Show last updated information if appropriate.

73. ADMIN SETTINGS

Create:

/admin/settings

Sections:

General

Store Information

Ordering

Delivery

Business Hours

Email

Branding

Do not show secret environment variable values.

Payment configuration UI may indicate whether required credentials are configured without revealing them.

Example:

Razorpay: Configured ✓

rather than displaying API secrets.

74. PRIVACY AND LEGAL PAGES

Create placeholder-but-professional pages:

/privacy

/terms

/refund-policy

/contact

/about

Do not fabricate legal guarantees.

Make policy text easy for the owner to edit.

75. FOOD-SPECIFIC INFORMATION

Architect product attributes so optional food information can later include:

- Allergens
- Ingredients
- Veg/non-veg
- Spice level
- Serving size
- Preparation time
- Nutrition information

Do not falsely claim products are allergen-free.

76. FOOD AVAILABILITY

Allow admin to quickly mark food as:

Available

Sold Out

Hidden

Provide quick availability toggle in admin product table.

Customer UI should immediately indicate:

"Sold Out"

and disable ordering.

77. OPTIONAL ORDER SCHEDULING

Design the architecture so the site can later support:

- ASAP delivery
- Scheduled pickup/delivery

This feature does not have to block the initial core implementation.

78. TESTING

Create tests for critical workflows.

Use:

- Vitest or Jest
- React Testing Library
- Playwright for E2E if practical

Critical tests:

Authentication

- registration
- invalid login
- protected admin route

Products

- customer can view available product
- admin can create/edit/archive product

Cart

- add item
- update quantity
- remove item

Checkout

- rejects unavailable item
- recalculates prices server-side
- rejects unsupported delivery location

Payment

- invalid signature rejected
- valid webhook marks payment paid
- duplicate webhook doesn't duplicate processing

Authorization

- customer cannot access admin APIs
- customer cannot view another customer's order

Reports

- date range correct
 - cancelled/refunded orders handled correctly
-

79. DEVELOPMENT SCRIPTS

Provide scripts such as:

`npm run dev`

`npm run build`

`npm run start`

`npm run lint`

`npm run typecheck`

`npm run test`

`npm run prisma:generate`

`npm run prisma:migrate`

`npm run prisma:seed`

Use exact scripts supported by the generated project.

80. README

Generate a thorough README.md containing:

1. Project overview
2. Features
3. Tech stack
4. Prerequisites
5. Installation
6. Environment variables
7. CockroachDB/PostgreSQL setup
8. Prisma setup
9. Database migration

10. Database seed
 11. Creating first admin
 12. Email configuration
 13. Payment gateway configuration
 14. Development payment mode
 15. Running locally
 16. Running tests
 17. Production build
 18. Deployment
 19. Security considerations
 20. Troubleshooting
-

81. INSTALLATION WORKFLOW

The final README should provide a straightforward workflow similar to:

1. Clone/install project.
2. Run:

```
npm install
```

1. Copy:

```
.env.example
```

to:

```
.env
```

1. Configure:

```
DATABASE_URL
```

1. Generate Prisma client:

```
npx prisma generate
```

1. Run migrations:

```
npx prisma migrate deploy
```

or the correct development command.

1. Seed database.

2. Start application:

npm run dev

Do not invent commands that are incompatible with the actual generated package configuration.

82. COCKROACHDB CONSIDERATIONS

Use the PostgreSQL-compatible CockroachDB database correctly.

Connection string:

```
DATABASE_URL="postgresql://srikanta:<ENTER-SQL-USER-PASSWORD>@dogged-ant-20102.jxf.gcp-asia-south1.cockroachlabs.cloud:26257/defaultdb?sslmode=verify-full"
```

The `<ENTER-SQL-USER-PASSWORD>` placeholder must be manually replaced in the environment configuration.

Never commit the actual password.

If Prisma or the PostgreSQL driver requires additional CockroachDB SSL handling, document the exact required steps.

Do not disable SSL certificate verification merely to make the connection work.

83. DEPLOYMENT

Make project deployable to a modern Node.js hosting service such as:

- Vercel
- Railway
- Render
- Fly.io

Database is remote CockroachDB.

Document required environment variables.

Do not expose environment secrets to frontend.

Set production:

```
NODE_ENV=production
```

Use HTTPS.

Explain webhook URL configuration for payment gateway.

Example concept:

`https://yourdomain.com/api/webhooks/payment`

84. PAYMENT PRODUCTION CHECKLIST

Before production, document that owner must:

- Create payment gateway merchant account.
 - Complete required KYC.
 - Obtain production API credentials.
 - Configure API credentials as environment secrets.
 - Configure webhook endpoint.
 - Set webhook secret.
 - Test successful payment.
 - Test failed payment.
 - Test abandoned payment.
 - Test duplicate webhook.
 - Test payment retry.
 - Verify that an order isn't confirmed without authenticated payment.
 - Switch off mock payment mode.
-

85. EMAIL PRODUCTION CHECKLIST

Document:

- Configure sender domain/email.
 - Configure SMTP/provider.
 - Verify sender identity if required.
 - Store credentials in environment.
 - Test successful order email.
 - Test provider failure.
 - Ensure email failure does not alter payment status.
-

86. CUSTOMER JOURNEY

Ensure complete customer workflow works from end to end:

Home → Browse Menu → Product → Add to Cart → Cart → Login/Register if required → Checkout → Address → Review → UPI payment → Provider confirmation → Server verification → Order confirmed → Email sent → Order success page → Account / My Orders → Track status

There must be no fake placeholder interactions in this core workflow.

87. ADMIN JOURNEY

Ensure complete admin workflow:

Admin Login → Dashboard → Add Category → Add Product → Product visible to customer → Customer orders product → Admin receives/sees order → Payment marked verified → Admin confirms order → Set Preparing → Set Ready → Set Out for Delivery → Set Delivered → Sale visible in reports

88. DEMO DATA

Seed enough meaningful demo data so UI is visually complete.

Include:

- 5 categories
- 15 products
- Example customers
- Example orders in different statuses
- Example paid/failed payments

NEVER seed fake production credentials.

89. DARK NEON DESIGN TOKENS

Create centralized design variables.

Example conceptual tokens:

--background --surface --surface-elevated --border --text-primary --text-secondary --primary --primary-foreground --accent --success --warning --danger

Create neon glow utilities sparingly.

Ensure all colors satisfy reasonable contrast.

90. ANIMATIONS

Use animations subtly.

Examples:

- Product cards gently raise on hover.
- Neon border intensifies on hover.
- Drawer transitions smoothly.
- Cart badge animates when item added.
- Dashboard cards fade in.
- Modal transitions.

Respect `prefers-reduced-motion`.

91. ADMIN TABLE COMPONENT

Create reusable table component supporting:

- Pagination
- Sorting
- Search
- Filter
- Loading
- Empty state
- Row actions

Use it for:

- products
 - orders
 - customers
 - payments
-

92. PAGINATION

Use server-side pagination for potentially large data.

Do not load every order/customer/payment into the browser.

Allow page sizes such as:

10 25 50 100

93. REPORT QUERY PERFORMANCE

For reports:

Use database aggregation.

Add indexes.

Avoid N+1 queries.

Use optimized joins/Prisma queries.

For expensive custom reports, create efficient service functions.

94. SOFT DELETE STRATEGY

Do not permanently delete business records that are required for historical reporting.

Products should generally be archived using:

deletedAt or isActive

Historical OrderItems must retain snapshots.

For truly unused products with no history, permanent deletion can be considered if intentionally implemented.

95. AUDITABILITY

Admin modifications should be traceable.

For sensitive changes log:

- Who made change
- What changed
- When
- Entity
- Previous value
- New value

Especially:

- Price
 - Product availability
 - Order status
 - Customer activation
 - Cancellation/refund
-

96. SERVER-SIDE AUTHORIZATION HELPERS

Create reusable authorization helpers.

Examples conceptually:

`requireUser()`

`requireAdmin()`

`getCurrentUser()`

Do not duplicate authorization logic inconsistently throughout the app.

97. SECURITY HEADERS

Configure appropriate headers such as:

- Content-Security-Policy where practical
- X-Content-Type-Options
- Referrer-Policy

- Permissions-Policy
- Strict-Transport-Security in production

Use framework-native security recommendations.

98. ENVIRONMENT VALIDATION

Validate required environment variables when the server starts.

Create a typed environment configuration using Zod or equivalent.

Fail clearly if critical production configuration is missing.

Do not leak secret values in error messages.

99. PRODUCTION VS DEVELOPMENT

Use clear environmental differences.

Development may use:

- seeded data
- mock payment
- local image placeholders

Production must:

- prohibit mock payments
- use real payment verification
- use configured email provider
- use HTTPS
- use proper secret management

Add safeguards so a production build cannot accidentally accept fake payments.

100. WHAT I EXPECT YOU TO GENERATE

Do not just describe the solution.

Generate the actual application code.

The project should contain:

- package.json
- TypeScript config
- Next.js config
- Tailwind configuration if required by installed version
- reusable UI components
- pages/routes
- authentication
- customer pages
- admin dashboard
- product CRUD
- category CRUD
- shopping cart
- checkout
- UPI payment abstraction/integration
- webhook endpoint
- email service
- order processing
- customer account
- reporting
- charts
- Prisma schema
- database migrations/setup
- seed script
- validation
- authorization
- error handling
- tests
- `.env.example`
- `.gitignore`
- README

Everything must be connected and functional.

101. IMPLEMENTATION PHASES

Work systematically.

Phase 1 — Architecture

Before writing large amounts of code:

- Define architecture.
- Define route map.

- Define database schema.
- Define authentication strategy.
- Define authorization strategy.
- Define payment architecture.
- Define email architecture.
- Define reporting architecture.
- Define folder structure.

Then implement.

Do not spend excessive output merely describing architecture instead of creating the application.

Phase 2 — Project Foundation

Implement:

- Next.js
 - TypeScript
 - Tailwind
 - UI system
 - Prisma
 - Database configuration
 - Authentication
 - Common layouts
-

Phase 3 — Customer Store

Implement:

- Home
 - Menu
 - Product pages
 - Search/filter
 - Cart
 - User registration/login
-

Phase 4 — Checkout

Implement:

- Addresses
- Checkout
- Order creation

- Server-side totals
 - UPI payment
 - Payment verification
 - Webhooks
 - Confirmation
 - Email
-

Phase 5 — Customer Account

Implement:

- Profile
 - Addresses
 - Order history
 - Order details
 - Status timeline
-

Phase 6 — Admin

Implement:

- Dashboard
 - Products
 - Categories
 - Orders
 - Customers
 - Payments
 - Settings
 - Audit logs
-

Phase 7 — Reporting

Implement:

- KPIs
 - Charts
 - Canned reports
 - Dynamic report builder
 - CSV export
 - XLSX/PDF if appropriate
-

Phase 8 — Hardening

Implement/check:

- Validation
 - RBAC
 - Transactions
 - Rate limiting
 - webhook idempotency
 - Logging
 - Security headers
 - responsive design
 - accessibility
 - performance
-

Phase 9 — Testing

Test complete customer and admin workflows.

Fix errors rather than leaving TODO comments.

102. IMPORTANT QUALITY REQUIREMENTS

Do NOT produce a superficial prototype.

Do NOT leave core functionality as:

TODO

"implement later"

pseudo-code

fake static JSON

fake admin data

fake order data

localStorage-only orders

localStorage-only authentication

The database must be the source of truth for:

- users
- products
- categories
- orders
- payments
- reports

The browser can use local/session state where appropriate for temporary UI behavior, but business data must persist server-side.

103. DO NOT MAKE THESE COMMON MISTAKES

Do not:

- Put database credentials in frontend code.
 - Expose payment secret key.
 - Store passwords in plaintext.
 - Trust cart price sent from browser.
 - Trust "payment successful" query string.
 - Confirm orders before verified payment.
 - Let customers query other customers' orders.
 - Allow customers to call admin APIs.
 - Permanently remove historical order data.
 - Use float for money.
 - Load every database row for reports.
 - Construct raw SQL from unrestricted report input.
 - Use localStorage as main database.
 - Skip webhook signature verification.
 - accidentally allow development mock payments in production.
 - send confirmation email before payment confirmation.
 - expose stack traces to users.
 - disable SSL verification just to fix database connection problems.
-

104. UX QUALITY CHECKLIST

Every major screen should include appropriate:

- loading state
- error state
- empty state
- success state

- validation state

Every button must do something real.

No dead buttons.

No broken links.

No placeholder navigation for primary features.

Forms must retain reasonable user input after recoverable validation errors.

105. FINAL VERIFICATION

Before declaring the project complete, verify:

Customer

- Can register
- Can login
- Can logout
- Can browse products
- Can search products
- Can filter products
- Can add to cart
- Can edit cart
- Can manage address
- Can checkout
- Can pay via UPI
- Payment verified server-side
- Confirmation email triggered
- Can view order
- Cannot see another customer's order

Admin

- Can log in
- Normal customer cannot access admin
- Can add product
- Can edit product
- Can archive/delete appropriately
- Can manage categories
- Can view orders
- Can update order status
- Can view customers

- Can view payments
- Can view canned reports
- Can build custom report
- Can export reports
- Can view dashboard analytics

Security

- DB password not in repository
- Payment secret not in browser
- Password hashed
- Authorization server-side
- webhook signature verified
- payment callback idempotent
- mock payments disabled in production
- input validated server-side

106. FINAL OUTPUT EXPECTATION

At completion provide:

1. Summary of architecture.
2. Complete source code.
3. Database schema.
4. Environment variable template.
5. Setup instructions.
6. Database migration instructions.
7. Seed instructions.
8. Admin account creation instructions.
9. Payment gateway setup instructions.
10. Email setup instructions.
11. Local development instructions.
12. Testing instructions.
13. Production deployment instructions.
14. Security checklist.
15. List of features implemented.
16. List of optional future enhancements.

If the environment allows direct file creation, create the complete folder structure and files rather than merely printing snippets.

After generating files:

- run dependency installation if permitted
- run TypeScript check
- run lint

- generate Prisma client
- run tests
- run production build

Fix any compile/build/type errors before calling the application complete.

The final project should look and behave like a real local food ordering product, not a tutorial demonstration.

107. BRAND PLACEHOLDERS

Until actual business information is provided, centralize these placeholders so they can be changed easily:

`BUSINESS_NAME = "Neon Bites"`

`TAGLINE = "Fresh Local Food. Bold Flavours."`

`BUSINESS_PHONE = "+91 XXXXX XXXXX"`

`BUSINESS_EMAIL = "orders@example.com"`

`BUSINESS_ADDRESS = "Your Local Address"`

Do not repeat these values manually throughout many components.

Create centralized store configuration.

108. OPTIONAL FUTURE-READY FEATURES

Do not allow these to delay the initial core implementation, but structure the system so they can be introduced later:

- WhatsApp order notifications
- SMS notifications
- Delivery staff accounts
- Kitchen dashboard
- QR-code menu
- Loyalty/reward points
- Coupons
- Referral codes
- Ratings and reviews
- Scheduled orders

- Table reservations
- Multiple branches
- Multi-language interface
- Kannada language support
- Hindi language support
- Push notifications
- PWA installation
- Invoice generation
- GST invoices
- Delivery tracking
- Google Maps integration
- Customer support chat
- WhatsApp support button

Core functionality must remain clean and modular so these enhancements do not require rewriting the whole project.

109. INITIAL UI DIRECTION

Use the public storefront as a premium food-first interface.

Think:

Dark charcoal background

+

Neon lime primary accent

+

Small cyan secondary accents

+

Large appetizing food imagery

+

Clean modern typography

+

High-contrast CTAs

+

Minimal sophisticated neon glow

Admin dashboard should feel like a premium SaaS analytics dashboard using the same visual language.

Avoid copying any particular brand.

Create an original design.

110. FINAL INSTRUCTION

Start building the application now.

Do not ask unnecessary questions.

Where business-specific data is missing, create centrally configurable placeholders instead of stopping development.

Make sensible production-grade decisions and document them.

Prioritize in this order:

1. Security
2. Data integrity
3. Correct order/payment flow
4. Authentication/authorization
5. Maintainable architecture
6. User experience
7. Responsive design
8. Visual polish

Build this as if it will actually accept real customer orders and real UPI payments in production.